

Advanced Technique And Tools For Software Development

*Sviluppo di una web application di un semplice
gestore di task*

Jacopo Vezzosi Mat. 7103062

Prof. Lorenzo Bettini



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Dipartimento di Matematica e Informatica "Ulisse Dini"

Università degli Studi di Firenze

A.A. 2024/2025

1 Applicazione

Ho sviluppato un semplice sistema per la gestione di task utilizzando Spring Boot. Il progetto si basa su un'unica entità: Task.

L'entità Task ha come attributi:

- id - String, generato automaticamente
- title - String
- description - String
- priority - int
- done - boolean

Per la gestione dei dati ho implementato una repository che estende MongoRepository al fine di utilizzare MongoDB per l'applicazione. Nella repository sono presenti alcune `findBy` scritte tramite le convenzioni di Spring. A scopo didattico, e per differenziarla dalle altre, `findAllTasksWithHighPriority` è stata implementata in modo custom tramite l'annotazione `@Query`.

La logica di business è concentrata nel service layer, che gestisce operazioni standard: Creazione, Lettura, Aggiornamento e Delete di un Task. Inoltre, è presente un metodo privato `dtoToEntity` che esegue il mapping dall'oggetto TaskDTO all'entità Task.

L'applicazione è accessibile sia tramite API REST, utili per integrazioni e test automatici, sia tramite interfaccia web, che consente di effettuare le operazioni descritte in precedenza nel livello service.

2 Tecniche e Framework

Per lo sviluppo dell'applicazione, è stato seguito l'approccio Test-Driven Development (TDD), scrivendo prima i test unitari e poi implementando il codice necessario per farli passare. Dopo i test unitari, sono stati eseguiti quelli di integrazione tra le componenti, considerato che i test unitari erano esaustivi, il comportamento dei test di integrazione era quello atteso e non ho riscontrato problematiche rilevanti nella loro implementazione. Infine per E2E tests mi sono concentrato sulle azioni eseguibili da parte di un utente, pertanto, partendo dalla home page: Creazione, Aggiornamento ed Eliminazione di un task, nel caso dell'Aggiornamento e dell'Eliminazione si è fatto uso di un metodo privato `postTask` per salvare il task tramite l'opportuna chiamata Rest. Per quanto riguarda l'esecuzione della build in Continuous Integration, ci sono state lievi problematiche, infatti, rispetto alla build locale, ci sono alcuni ritardi nel caricamento dei contenuti della pagina e sono quindi state inserite delle `wait` opportune.

Di seguito si riportano i framework utilizzati:

- **Ambiente di sviluppo**
 - Eclipse (Spring Tools for Eclipse), Maven
 - Spring Boot
- **Docker & Database**
 - MongoRepository
 - Flapdoodle Embedded Mongo, MongoDB
 - Docker, Docker Compose, Docker Maven Plugin
 - Spring Profiles (configurazione MongoDB per IT)
- **Web e View**
 - Thymeleaf
 - HTML
- **Testing (Unit, Integration, E2E)**
 - JUnit 4 (con JUnit Vintage Engine)
 - Mockito, AssertJ, Hamcrest, JSONAssert, JsonPath

- Selenium, HtmlUnit
- Spring Boot Test, Spring Test

- **Quality Assurance & Code Analysis**

- PIT (Mutation Testing)
- Jacoco (Code Coverage)
- SonarQube, SonarCloud
- Coveralls

- **Version Control & Continuous Integration**

- Git, GitHub
- GitHub Actions (Continuous Integration)

3 Design, Sviluppo e Testing

Nel design dell'applicazione ho dato priorità a modularità e facilità di manutenzione. L'adozione dell'architettura Controller → Service → Repository risponde all'esigenza della separazione delle responsabilità: il controller gestisce le richieste e l'interazione con il sistema, il service gestisce la logica applicativa, mentre la repository si occupa dell'accesso ai dati nel database.

Lo sviluppo è proceduto seguendo un approccio bottom-up: ho iniziato definendo il livello di persistenza con la repository, per poi implementare il livello service contenente la logica di business, e infine ho realizzato i controller (Rest e Web) che espongono le funzionalità all'esterno. Questa progressione ha permesso di testare e consolidare ogni strato prima di procedere con quello successivo.

Dal punto di vista del testing, ogni componente dell'applicazione è testato in isolamento seguendo l'approccio TDD. In particolare, gli unit test sono stati progettati sfruttando la dependency injection, che permette di fornire a ciascun componente solo le dipendenze necessarie, sostituendo eventualmente le altre con istanze Mock o Spy. In questo modo, ogni classe viene testata indipendentemente dal comportamento delle altre, garantendo che la logica interna funzioni correttamente. Inoltre, ho utilizzato un database embedded Mongo, con una strategia simile al materiale dell'esame che però utilizza h2 perché l'esempio mostra un approccio implementativo SQL-Oriented. Infine, per le stesse motivazioni appena descritte, ho utilizzato DataMongoTest invece di DataJpaTest.

Gli Integration Test, invece, verificano la corretta collaborazione tra i diversi componenti, interagendo con un Docker container contenente un database reale MongoDB. Il container viene avviato automaticamente nella fase di pre-integration test e viene fermato nella fase di post-integration test. Vista la presenza del database embedded mongo, ho creato un profilo spring `application-mongodb.yml` con l'intento di escludere quest'ultimo ed utilizzare il database MongoDB presente nel container, questo profilo viene attivato direttamente all'interno delle classi di test interessate, cioè tutte le classi riguardanti i test di integrazione, tramite l'annotazione `@ActiveProfiles("mongodb")`.

Infine, nei test E2E, l'applicazione web parte automaticamente dopo aver avviato il container, vengono eseguiti scenari completi che simulano l'interazione reale dell'utente con l'interfaccia: partendo dalla home page, si effettuano creazione, aggiornamento ed eliminazione di un Task. In questo caso non è necessario definire `@ActiveProfiles("mongodb")` poiché i test sono plain JUnit e vengono eseguiti

sull'applicazione già avviata, e quest'ultima non viene influenzata dal database embedded perché nel POM è stato definito `<scope>test</scope>`.

4 Tecnologie High Rating e Difficoltà Riscontrate

Tra le normali difficoltà incontrate nello sviluppo del progetto, se ne riportano alcune degne di nota inserendo una spiegazione esaustiva dove necessario.

4.1 MongoDB

Rispetto al materiale dell'esame si sono rese necessarie ulteriori indagini su come implementare e configurare tutti gli aspetti differenti che Mongo porta con sé, rispetto a quanto fatto anche nei progetti precedenti con database SQL-Oriented. Per esempio l'utilizzo di `@DataMongoTest` che, a differenza di `@DataJpaTest` presenta test non transazionali. Un'altra differenza, rispetto alle configurazioni dell'esempio e a quanto fatto nei progetti precedenti, è la configurazione di MongoDB che, al fine di utilizzarlo, richiedeva l'esclusione esplicita del db embedded mongo, per fare ciò, come accennato in precedenza, nella directory `test/resources` si è definito un profilo appropriato.

4.2 Delete

Per implementare l'operazione di eliminazione di un Task, non si sono presentate problematiche fino al web controller, infatti per utilizzare il `@DeleteMapping` è stato necessario specificare nei file `application.properties` la stringa `spring.mvc.hiddenmethod.filter.enabled=true` perché la form non supporta nativamente l'operazione Delete. Un'alternativa poteva essere utilizzare un mapping differente supportato come `@GetMapping`, tuttavia, risultava poco coerente e ho preferito l'altra strada.

4.3 Data Transfer Object

Un ulteriore aspetto, è stato l'utilizzo di un DTO. Anche se in questo caso non ci sono reali dati sensibili, ed i campi dell'entità Task erano utilizzati nella loro totalità, SonarQube segnala un problema di sicurezza quando si passa un'entità ad un mapping Post, pertanto ho creato una classe TaskDTO, equivalente all'entità Task. Il mapping tra il DTO e l'entità, come accennato precedentemente, è implementato nel service; non viene eseguito il mapping dell'id per ragioni implementative, infatti nell'operazione di aggiornamento viene settato l'id del vecchio task, mentre nell'operazione di creazione l'id è null.

5 Istruzioni Riproduzione Build

Per l'esecuzione della build, posizionarsi nella directory principale dove è presente il file `pom.xml`, avviare Docker (i container partono automaticamente durante la build). In caso l'esecuzione della build con i mutation-testing risultasse troppo lenta, commentare `targetClasses` e `targetTests` relativi ai controllers, dato che la logica è contenuta nel service, in entrambi i casi nessun mutante sopravvive.

Per l'esecuzione di Unit Test, Integration Test, Code Coverage e Mutation Testing:

```
mvn verify -Pjacoco,mutation-testing
```

Per l'esecuzione solitaria degli E2E Tests:

```
mvn verify -Pe2e-tests
```